Laboratorio: Definición y uso de WebSocket

Los estudiantes, agrupados en equipos de hasta 3 integrantes deberán desarrollar el siguiente laboratorio, tomando como base el proyecto disponible en Educandus (***WebSocket*** desarrollado en Java). Se les pide realizar un análisis y comparación de las tres implementaciones, planteando diferencias ya sea en su estructura lógica o de comunicación.

Además, deberán extender el proyecto en ***al menos 2*** de las siguientes opciones de desarrollo:

Opción 1: Mensajería Privada y Protocolo en JSON (Diseño de Software / Capa de Aplicación)

Regla de Negocio: Abandonar los mensajes de texto plano e implementar un protocolo basado en tramas *JSON* estructuradas.

Requerimientos:

- El payload debe contemplar: tipo (`"PUBLICO", "PRIVADO", "LISTA_USUARIOS"`), origen, destino y mensaje.
- Permitir comandos de susurro por consola o interfaz, por ejemplo: ***`@pedro`*** `hola` para enviar un mensaje unicast exclusivo entre dos sesiones sin emitirlo a toda la sala.

Permite poner en práctica: Serialización/deserialización, manejo de colecciones indexadas por identificador de usuario y diseño de protocolos a nivel de aplicación.

Opción 2: Lista de Presencia Activa en Tiempo Real (Sistemas Distribuidos)

Regla de Negocio: Mantener un panel lateral en el cliente web que muestre en tiempo real la lista de todos los usuarios actualmente conectados.

Requerimientos:

- Cada vez que un usuario entra (`@OnOpen`) o se va (`@OnClose` / error), el servidor debe generar un mensaje de control con el arreglo de nombres activos y difundirlo a todos los clientes.
- El frontend debe refrescar dinámicamente un elemento `<ul>` con la lista de participantes.

Permite poner en práctica: Sincronización de estado entre clientes y backend, manejo de eventos de ciclo de vida del socket y actualización de DOM reactiva.

Opción 3: Detección de Caídas Silenciosas mediante Heartbeat / Ping-Pong (Resiliencia en Redes)

Regla de Negocio: Resolver el problema de los sockets huérfanos (half-open connections) que ocurren cuando un cliente pierde conectividad abruptamente (Por ejemplo: desconexión de Wi-Fi o suspensión de la máquina) sin disparar un `close()`.

Requerimiento:

- Implementar un mecanismo de heartbeat donde el servidor (o cliente) envíe tramas de control `PING` cada 10 segundos.
- Si un cliente no responde con `PONG` en un intervalo límite (ej. 30 segundos), el servidor debe forzar `session.close()` y liberar la memoria del mapa concurrente.

Permite poner en práctica: Comprensión de los límites de TCP ante fallos de enlace físico, timeouts de red y recolección de recursos zombi.

Opción 4: Salas de Chat Múltiples / Canales Temáticos (Concurrencia y Estructuras de Datos)

Regla de Negocio: Ampliar el servidor para permitir múltiples salas simultáneas, por ejemplo: `#general, #ayudantia, #random`.

Requerimiento:

- Modificar el handshake para recibir tanto el nombre como la sala (`?nombre=Juan&sala=general`).
- En el servidor, reemplazar el mapa plano de sesiones por una estructura agrupada: `Map<String, Set<Session>>`.
- El broadcast debe aislarse estrictamente para emitir solo a los clientes pertenecientes a la misma sala.

Permite poner en práctica: Concurrencia segura multihilo (`ConcurrentMap`, conjuntos sincronizados), aislamiento de dominios de broadcast y escalabilidad lógica.

Entrega:
El laboratorio tendrá como plazo límite de entrega el próximo viernes 30 de Octubre. Como evidencia de su trabajo, deberán subir un avance a Educandus, hoy antes del término de la sesión.